Modelling (part 1)

CSE 362
Software Development and Professional Practice
Winter 2023
© Royal Military College of Canada

Modelling = Analysis + Design

- During software analysis and design, a lot of communication takes place.

- This communication is supported by all kinds of notations to convey its message, e.g., a sketch of a screen layout.

- The most common notations to support communication involve some sort of *box-and-line diagrams*.

- Sometime these diagrams have very informal semantics.

- While being drawn, these loose semantics may not be a problem. But the next day confusions arise.

- An alternative is to use diagrams with precise semantics…

What's good about using formal diagrams:

- Reduced to no confusion as to the meaning.
- Diagrams can be subjected to all kinds of checks.
- Tools may generate the diagrams.
- Tools may read the diagrams.
- And in some cases tools may generate executable code from the diagrams.

In this lecture, we discuss a number of semiformal modeling notations.

Most are box-and-line diagrams

Some use a more textual layout.

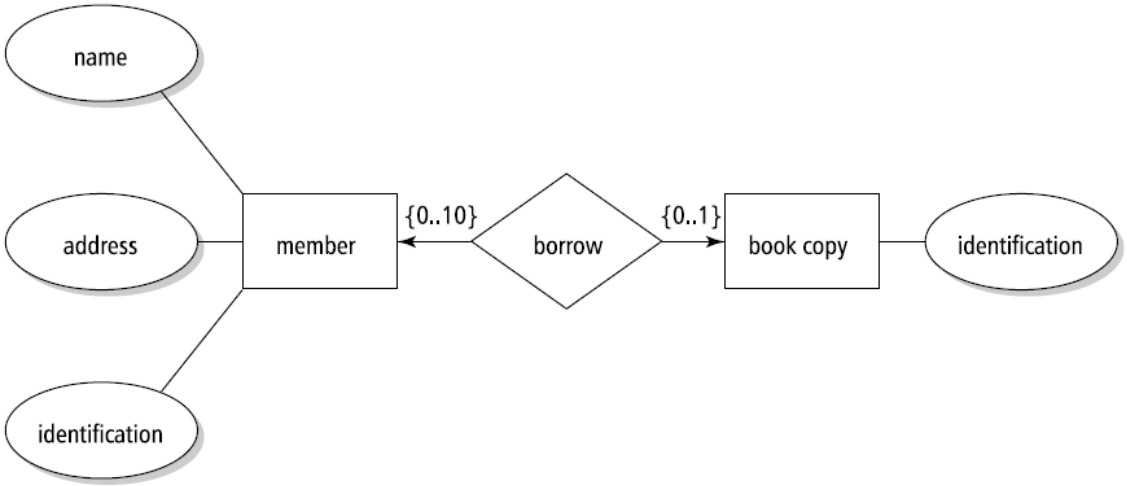
Classic Modeling Techniques

- Entity-Relationship Modeling (ERM)
- Finite State Machines (FSM) are used to model states and state transitions.
- Data Flow Diagrams (DFD) model a system as a set of processes and data flows that connect these processes. It is the notation used in data flow design.
- CRC cards are a simple requirements elicitation tool.

Entity-Relationship Modeling (ERM)

- The basic ingredients of ERM are

entity	distinguishable object of some type
attribute value	piece of information (partially) describing an entity
attribute	type of a set of attribute values
relationship	association between two or more entities



Finite State Machines (FSM)

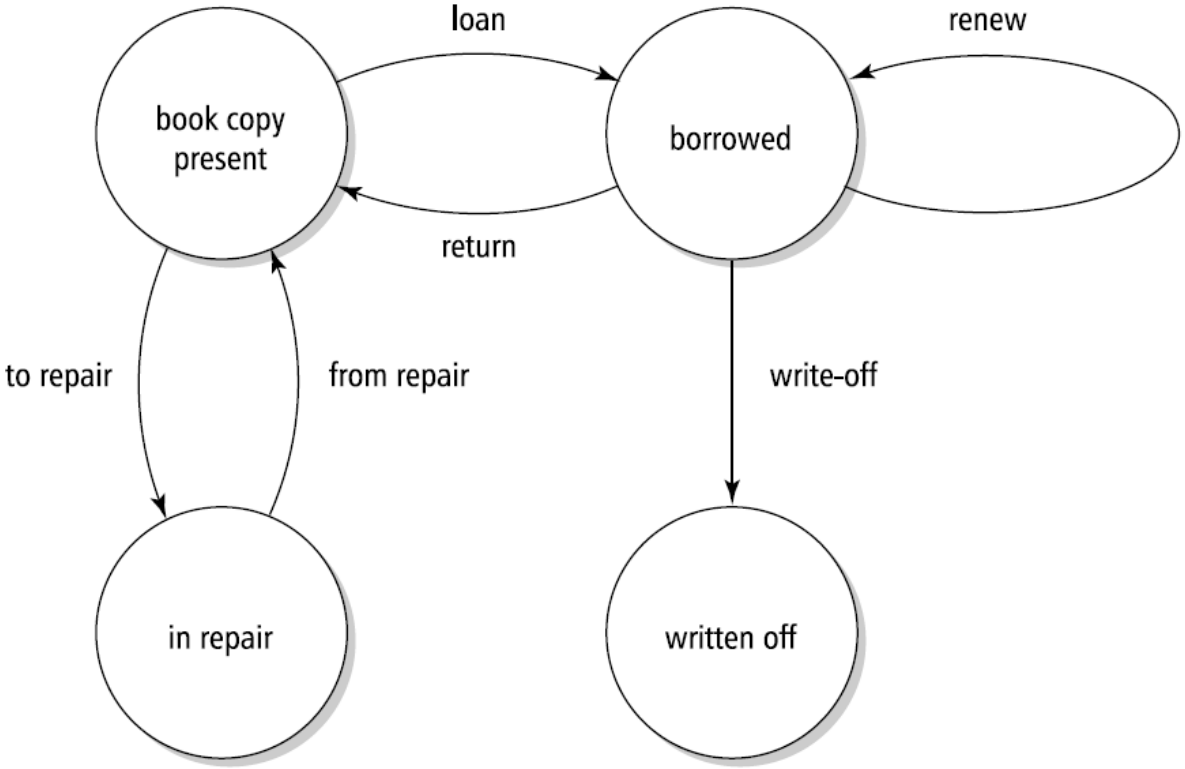
At any one point in time, a library is in one of a (vast) number of possible states. The state of the library can be expressed in terms of things like:

- the collection of titles available,
- the collection of titles ordered but not yet received,
- the collection of library members,
- the balance of the account from which acquisitions are paid.

Any action occurring in the library, be it the return of a book or the appointment of a new employee, transforms the current state s into a new state s'.

Finite State Machines (FSM)

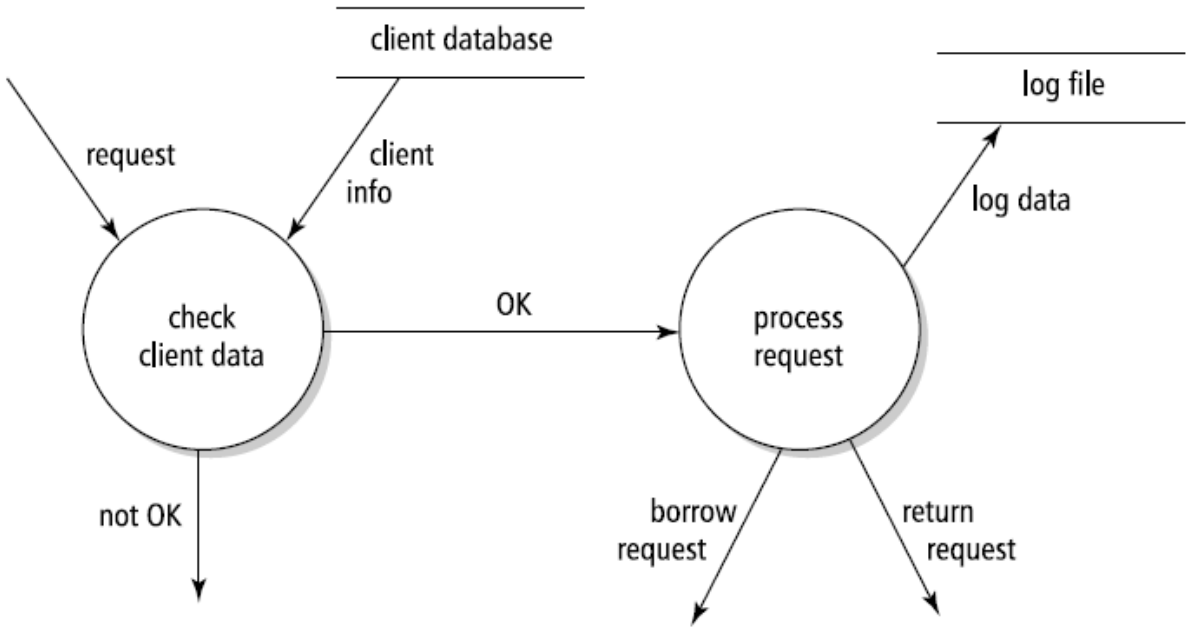
- **Finite State Machine** (FSM) is a formalism to model a system in terms of *states* and *transitions between states*.



Data Flow Diagrams (DFD)

- a functional decomposition with respect to the flow of data.

- A component (module) is a black box which transforms some input stream into some output stream.



CRC cards

- CRC stands for Class -- Responsibility -- Collaborators.

Class Reservations	Collaborators • Catalog • User session
Responsibility • Keep list of reserved titles • Handle reservations	

Objects and Related Stuff

Object in Analysis and Design

"analysis and design is about modeling real-world objects."

"Analysis and design is about identifying reusable components and building their inheritance hierarchy."

What is an Object?

An object is a conceptual model of some part of a real or imaginary world.

Object = identity + variables + operations

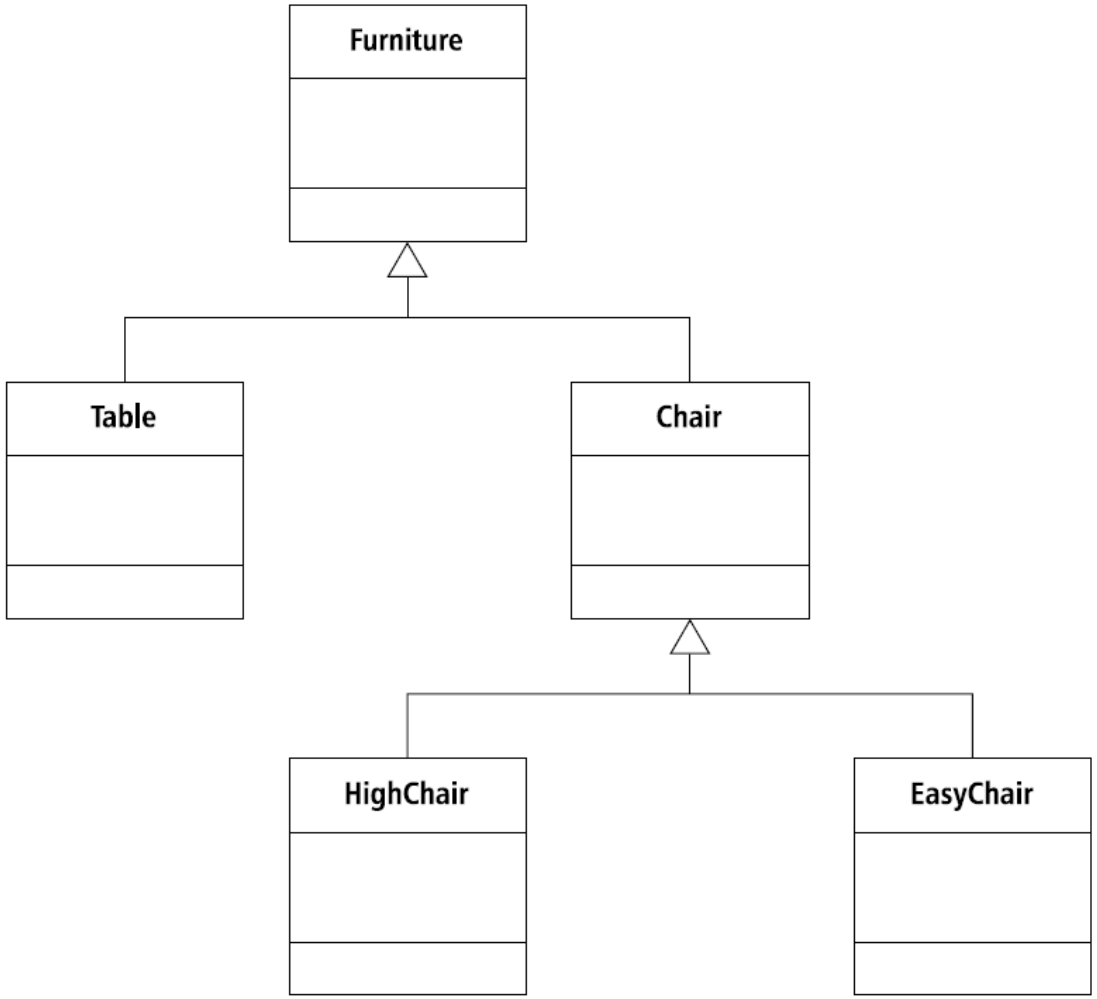
Or Object = identity + state + behaviour

Relationships between Objects

- The major types of relations between objects:

Relation	Example
Specialization/Generalization, is-a	Table *is-a* Furniture
Whole-part, has	Table *has* TableTop
Member-of, has	Library *has* Member

Inheritance Hierarchy



Let’s practice this part

Where to Start the Analysis?

- Focus on the User Stories and the resulting Functional and Non-Functional Requirements.
- A *traditional "User Story"* is like the script of a TV commercial that describes a typical scenario and use of your system.
- Many User Stories can be abstracted into a smaller number of Use Cases.
- Requirements are extracted from the Use Cases.
- Look over the RAD to identify the Objects that are part of your system, so you can use a consistent set of names in your analysis.

Identifying Object Hierarchies

- If you are using an OOP development language, breaking the system down into objects is really helpful!

- Where possible, create object hierarchies that will:
  - Help to indicate which objects are more abstract than others.
  - Allow you to simplify construction by referring to more abstract objects.
  - Identify relationships between objects.
  - Help to identify gaps.

Classifying Objects

- Take your set of participating objects, located by an examination of the RAD and user stories, and separate them into three categories:
  - Entity Objects – Persistent information tracked by system.
  - Boundary Objects – Where the actor interacts with the system.
  - Control Objects – Makes it work!

Aside – "Actors"

- This is a traditional term that really just means "Users" or a "User Group".
- But it sometimes can include another separate system that is accessed by your system.
- You might have different types of users, that are characterized by the different way they interact with your system: Landlords, Tenants, Administrators, Players, Students, Professors, etc.
- Each would be represented by an "Actor".
- Any system must have at least one Actor, and might have more than one.

Aside - Mapping Natural Language to Model Components – Examples

As Language	Model Component	Example
Proper noun	Instance	Alice
Common noun	Class	Player
Doing verb	Operation	Creates, submits
Being verb	Inheritance	Is a kind of
Having verb	Aggregation	Has, consists of
Modal verb	Constraints	Must be
Adjective	Attribute	Game score

FRIEND System Example

- This system is to be used by dispatchers and emergency responders to create an emergency response and then to record that response in a database.

- Start by looking over one "Use Case" that describes a typical use of the system.

ReportEmergency Use Case Example

- **Use case name**: **ReportEmergency**
- **Participating Actors**:
  - Initiated by FieldOfficer
  - Communicates with Dispatcher
- **Flow of Events**:

1. FieldOfficer activates "Report Emergency" function on her laptop.
2. FRIEND responds by presenting a form to the FieldOfficer. The form includes an emergency type list to choose from, location details, incident description, resources requested and hazardous materials involved option.
3. FieldOfficer provides a description of the situation, the emergency level, location, and the type of emergency, along with possible responses. Once complete, the FieldOfficer submits the form along with a request for a specific response. The minimum required information is emergency type and description.

ReportEmergency Use Case Example

4. FRIEND receives the form and notifies the Dispatcher using a pop up dialog.

5. Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. All information in the report submitted by the FieldOfficer is included in the Incident. The Dispatcher selects a response by allocating resources to the Incident (using the AllocateResources use case) and acknowledges the report by sending a Acknowledgement to the FieldOfficer. The Acknowledgement indicates to the FieldOfficer that the EmergencyReport was received, and Incident created and resources were allocated to the Incident, along with the type of resources and the ETA.

6. FRIEND displays the acknowledgement and response to the FieldOfficer.

ReportEmergency Use Case Example

- **Entry Condition**:
  - The FieldOfficer is logged into FRIEND.
- **Exit Conditions**:
  - The FieldOfficer has received an acknowledgement and the selected response from the dispatcher, OR
  - The FieldOfficer has received an explanation of why the transaction could not be processed.
- **Quality Requirements**:
  - The FieldOfficer's report is acknowledged within 30 seconds.
  - The selected response arrives no later than 30 seconds after it is sent by the Dispatcher.

Use Cases vs. Agile User Stories

- Agile user stories are much shorter!
- But, you probably have more user stories than you would have use cases.
- A use case contains information on what the system is doing along a time line, it does not just describe a requirement.
- A use case can also contain non-functional requirements.

Equivalent Agile User Stories?

- As an emergency supervisor, I want to be able to collect, analyze and review all data for emergency responses through the use of a database, so problems can be corrected and system improvements can be made.

Equivalent Agile User Stories?

- As a field officer I want to be able to provide emergency information to the dispatcher by filling out a form on my laptop, so that all information is supplied, recorded and is not ambiguous.

- As a field officer I want to receive a response from the dispatcher that contains specifics about the response and the ETA so I know what to expect and I can judge if I think the response is adequate.

Equivalent Agile User Stories?

- As a dispatcher I want to receive notifications of new emergencies on my computer.
- As a dispatcher I want to be able to have a real-time view of the location and availability of my emergency response assets.
- As a dispatcher I want to be able to allocate assets to the new emergency and record this allocation.
- As a dispatcher I want to send an update to the field officer containing the response information so that they are informed of the details.

+ Non-Functional Requirements

- The field officer has a laptop running the system.

- The field officer's original notification is received and acknowledged by the dispatcher within 30 seconds.

- The response details created by the dispatcher is received by the field officer within 30 seconds.

Use Cases vs. Agile User Stories, Cont.

- To make a time to completion estimate for a user story, the team must eliminate any ambiguity in the story and discover non-functional requirements. Maybe break it down into separate sub-tasks?...

- As they discuss the user story, they seem to actually be building a use case, but normally they would not document it.

- And, they would get to work on it right away!

Back to Object Names

- Go over the Object types in more detail, classify them, and then start to quantify how they interact along a timeline

Entity Objects

- Terms from the glossary.
- Recurring nouns from use cases (“Incident”).
- Real world entities that the system needs to track (“FieldOfficer”, “Dispatcher”, “Resources”).
- Real world activities that must be tracked (“EmergencyOperationsPlan”).
- Data sources or sinks (“Printer”).

Boundary Objects

- UI controls that the user needs to initiate a use case ("ReportEmergencyButton").
- Forms needed to obtain information from the user ("EmergencyReportForm").
- Messages sent by the system to the user ("AcknowlegementNotice").
- Separate actor terminals ("DispatcherTerminal").

- Leave more detail of components to UI prototypes or "mock-ups".
- Continue to use application domain terms.

ReportEmergency Objects

- Entity:

Dispatcher, EmergencyReport, FieldOfficer, Incident, Acknowledgement

- Boundary:

AcknowledgementNotice, DispatcherTerminal, ReportEmergencyButton, EmergencyReportForm, FieldOfficerTerminal, IncidentForm

- Control:

ReportEmergencyControl, ManageEmergencyControl

Next part of this practice

- How to represent the interaction of these objects both statically and along a timeline using UML Diagrams.

- Now back to some theory

The Unified Modeling Language (UML)

UML

- The most widely used notation for both requirements engineering and design.

- The current version is known as UML 2 – 13 diagrams.

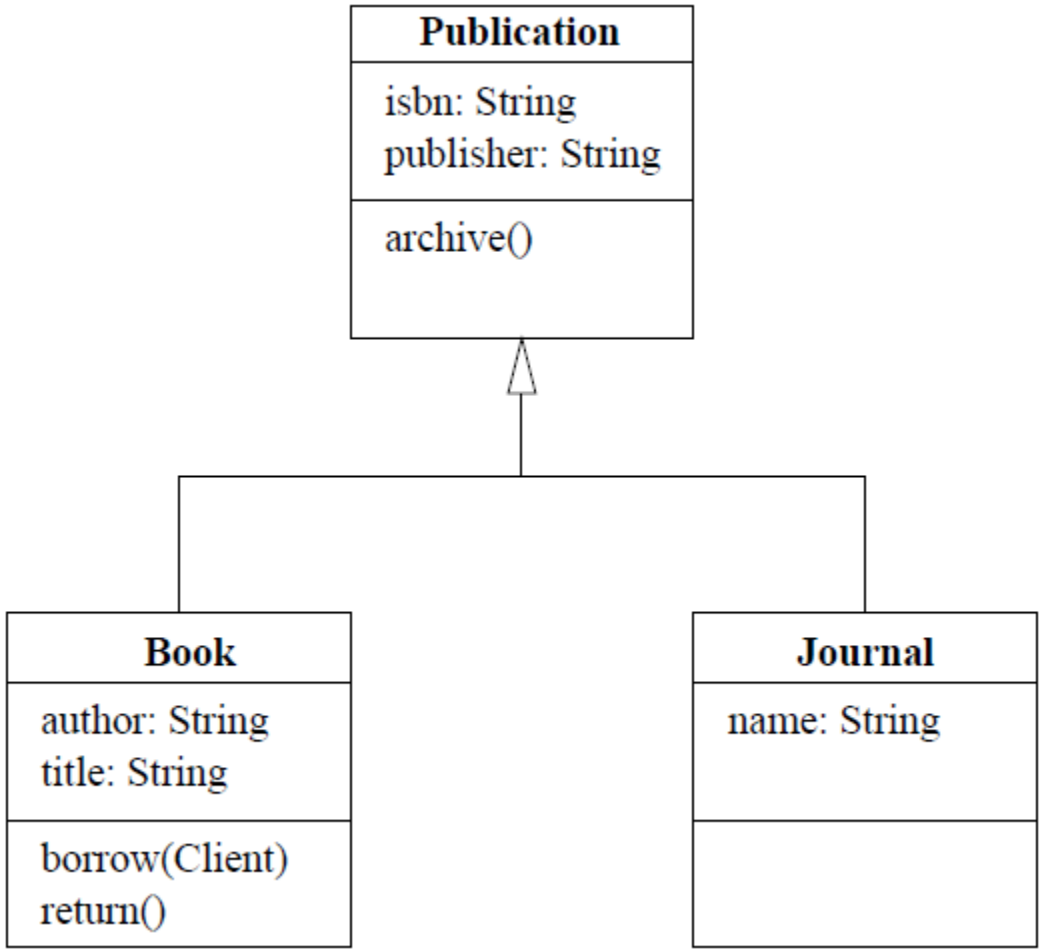
- Some of the diagrams give a static (S) view. Other diagrams give a dynamic (D), or behavioral view, i.e. they show what happens when the system is executed.

Diagram	Description
Activity (D)	To model business processes, workflow, procedural logic. Similar to flowcharts, but activity diagrams support parallelism, like in Petri nets
Class (S)	To model classes, their features and relationships. See section 10.3.1
Communication (D)	To model the flow of messages between instances of classes. Very similar to the sequence diagram. See section 10.3.4
Component (S)	To model a set of components and their interrelationships (through interfaces). See section 10.3.5
Composite structure	To model the internal dynamic structure of a class
Deployment	To model the physical layout, i.e. the assignment of system elements to hardware elements
Interaction overview (S)	Combines activity diagrams and sequence diagrams
Object (S)	To model objects and their relationships at some point in time; also known as instance diagram
Package (S)	To model the grouping of elements into packages
Sequence (D)	To model the order in which messages are exchanged between instances of classes. See section 10.3.3
State machine (D)	To model the states in which an object can be, and the transition between states. See section 10.3.2
Timing (D)	To model state changes of an object over time
Use case (D)	To model use cases. See section 10.3.6

The Class Diagram

- Depicts the static structure of the system.
- A graph where the nodes are objects (classes) and the edges are relationships between objects.
- By decorating the edges, many kinds of relationships can be modelled. (*generalizations* and *associations*)

The Class Diagram: Generalization



The Class Diagram: Association

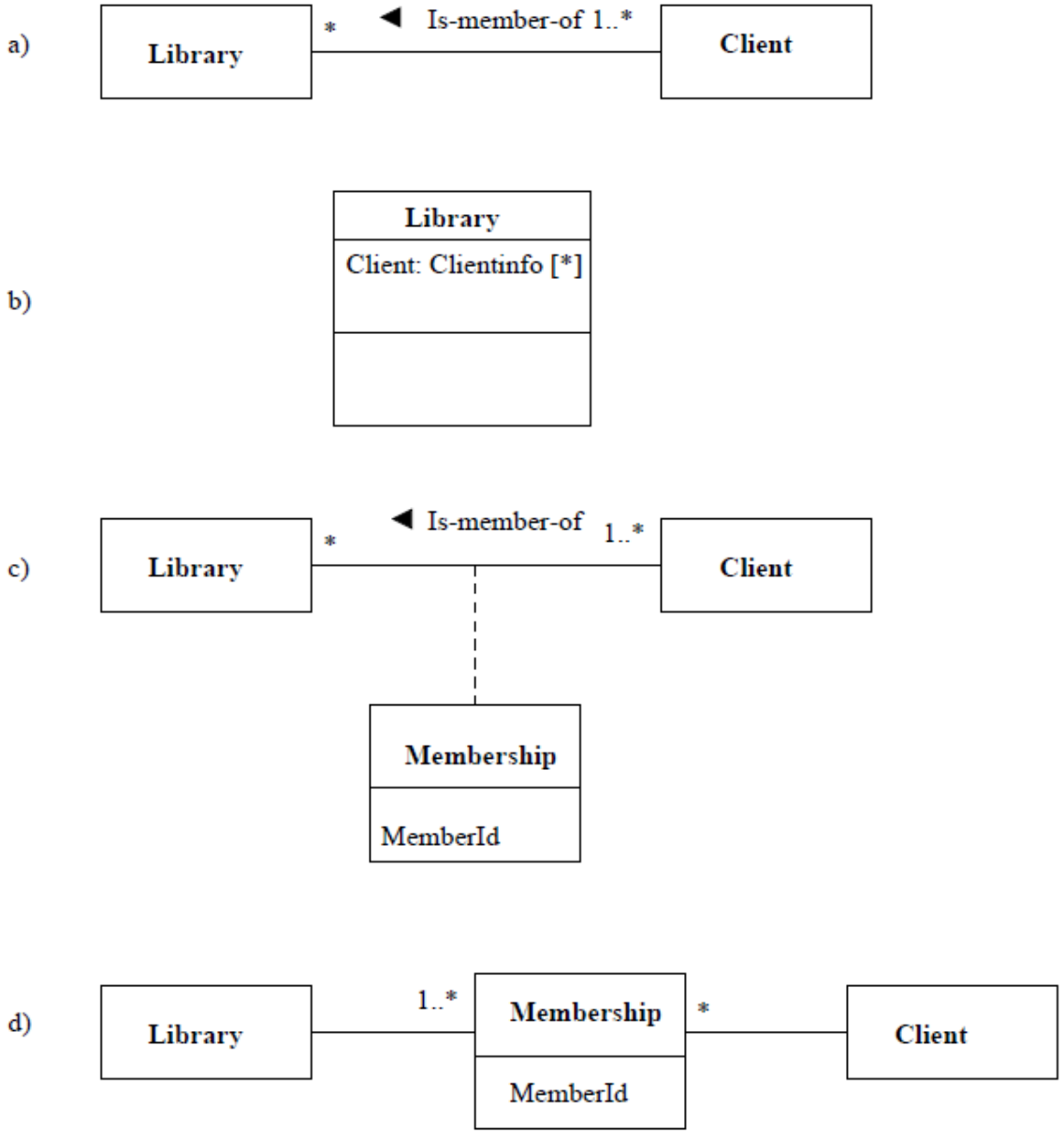


Figure 10.10 UML class diagram: (a) association, (b) association as attribute, (c) association class, (d) association class as a full class

The Sequence Diagram

- A UML Sequence Diagram re-creates a use case using its objects.

- They consist of the actors, the boundary, control and entity objects, with interactions shown along a time line.

- Can be pretty complicated!

The Sequence Diagram

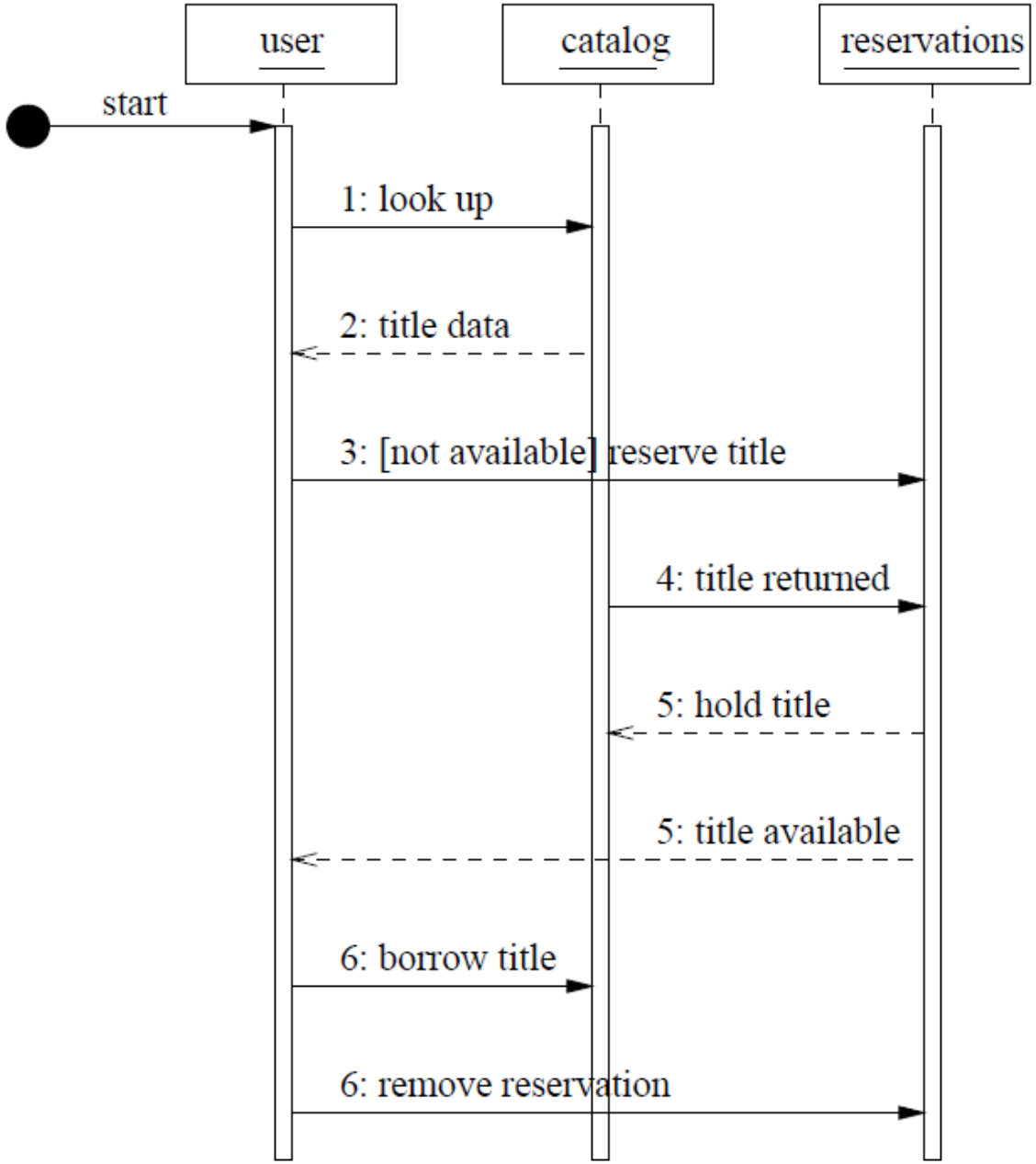
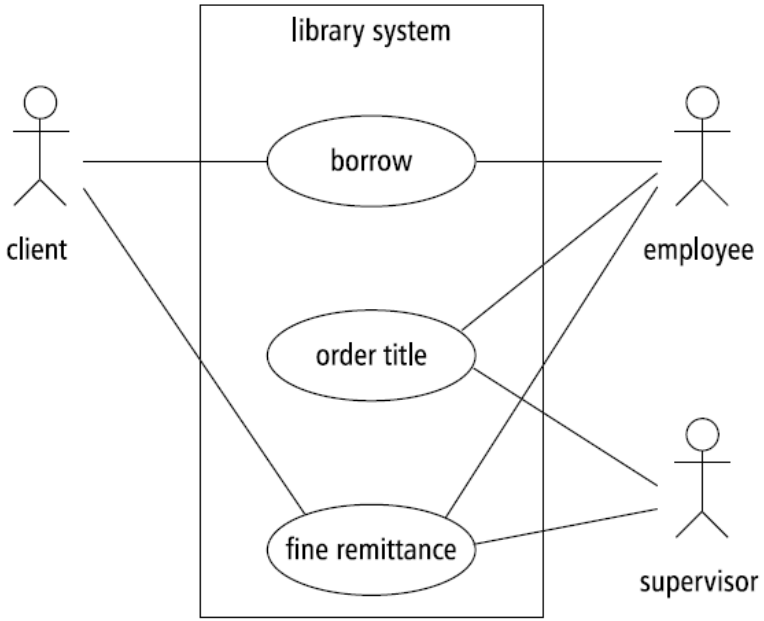


Figure 10.15 UML sequence diagram: reserving a title

The Use Case Diagram

- A use case can be documented in various ways: as narrative text, formally using pre- and postconditions, for example, or graphically as in a state transition diagram.
- The **use case diagram** provides an overview of a set of use cases.